THM TECHNISCHE HOCHSCHULE MITTELHESSEN
CAMPUS FRIEDBERG
MND
Mathematik, Naturwissenschaften und Datenverarbeitung

Conflict-Free Replicated Datatypes

Fortgeschrittene Datenbanktechnologien

Agenda

1. Motivation & Problemstellung
2. Textbearbeitung
3. Einführung in CRDTs
4. Funktionsweise von CRDTs
   - a. Verschachtelungen & Anomalien
   - b. Verschiebung von Elementen in Listen
   - c. Verschiebung von Knoten in Baumstrukturen
   - d. Performance von CRDT-Algorithmen
5. Zusammenfassung
(6) Übung

Schonmal etwas von CRDT gehört?

Gut, ich auch nicht.

Zugrundeliegende Begriffsdefinitionen

Concurrent (gleichzeitig):	Zwei Ereignisse sind gleichzeitig, wenn sie ohne Kenntnis voneinander eintreten
Conflict-free (konfliktfrei):	Keine exklusiven Schreibrechte für Operationen notwendig. Gleichzeitige Aktualisierungen werden erkannt und Konflikte können automatisch gelöst werden.
Commutative Replicated Data Type (CmRDT):	Neue Bezeichnung = „Operation-Based CRDTs“ (Op-Based)
Convergent Replicated Data Type (CvRDT):	Neue Bezeichnung = „State-Based CRDTs“ (zustandsbasiert/konvergent)
Convergence (Konvergenz):	Korrektheitseigenschaft: Wenn zwei Replikate denselben Satz von Aktualisierungen verarbeitet haben, müssen sich diese Replikate im selben Zustand befinden.
Last writer wins (LWW):	Aktualisierungen mit höheren Zeitstempeln überschreiben die Werte von Aktualisierungen mit niedrigeren Zeitstempeln.
Optimistic replication (opt. Replikation) :	Ein Replikationsansatz, bei dem ein Replikat Aktualisierungen lokal verarbeiten kann, ohne auf die Kommunikation mit anderen Replikaten zu warten.
Add-wins set (AWSet):	Ein Mengen-Datentyp, bei dem Hinzufügungen Vorrang vor Entfernungen haben.

Motivation & Problemstellung

Motivation & Problemstellung

- Konsistenz ist ein grundlegendes Merkmale verteilter Systeme
  - World Wide Web
  - Peer-to-Peer
  - Cloud-Computing-Plattformen
  - Datenbanken

➡ Nutzer haben keine permanente Netzwerkverbindung

- Use Cases und Implementierungen von CRDTs
  - Amazon verwendet CRDTs, um deren Warenkörbe zu synchronisieren
  - Riat Games nutzt CRDTs für den In-Game-Chat seines Spiels League of Legends
  - Rovio (Angry Birds), SoundClound und TomTom verwenden CRDTs

Textbearbeitung

Kollaborationssoftware

Abb. 1

Abb. 2

Abb.1: https://transkriptor.com/de/sprach-zu-text-google-docs/

Abb. 2: https://blog.acolad.com/de/produkt-und-dienstbeschreibung-von-office-365

Kollaborationssoftware

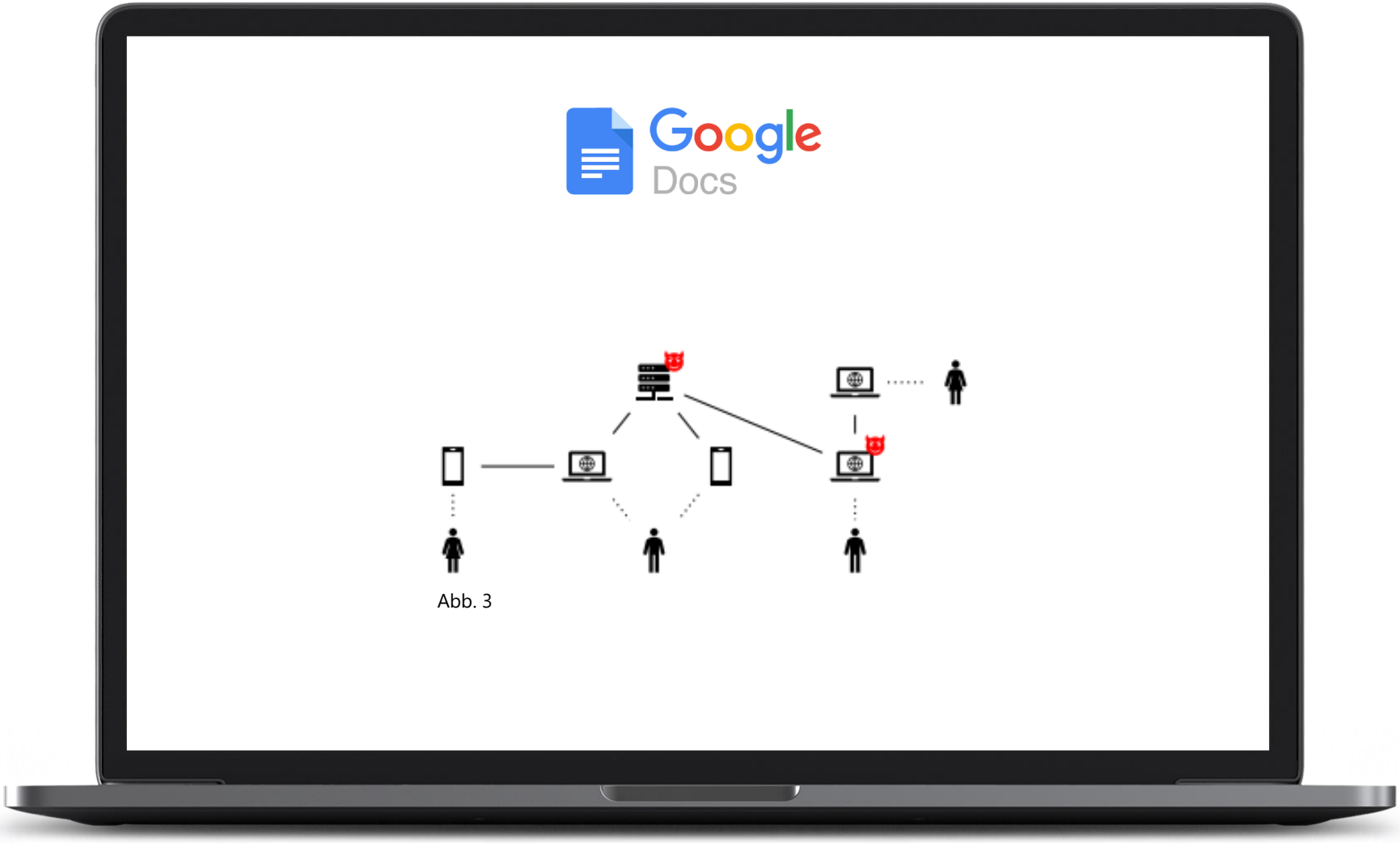
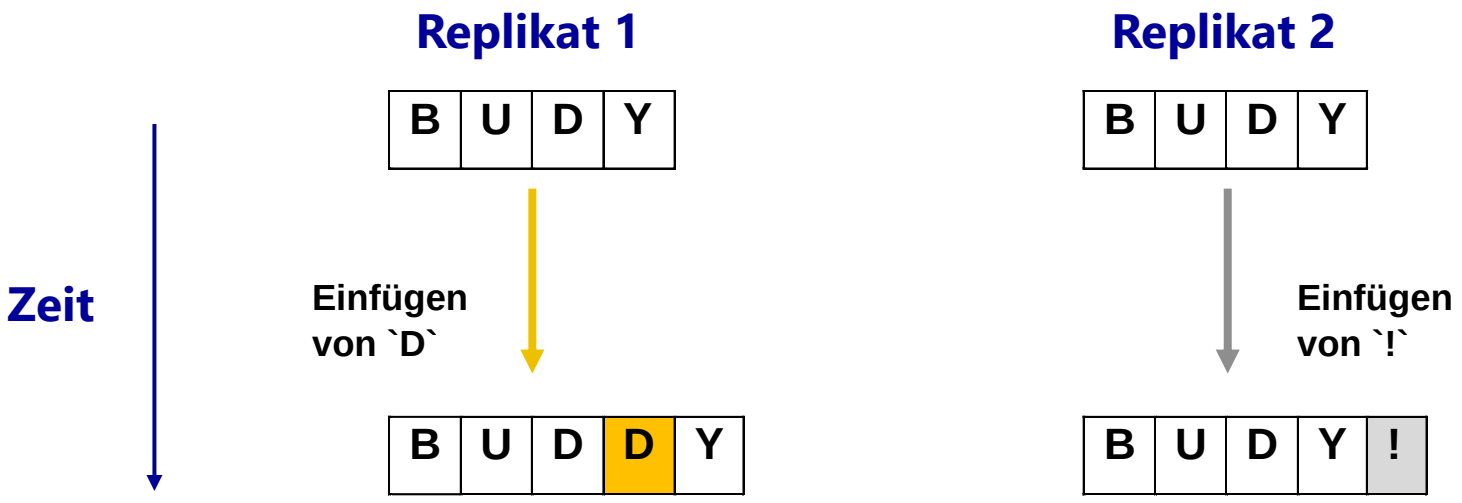


Abb. 3

Abb. 3: https://kristofjannes.com/papers/dicg22.pdf, S. 1

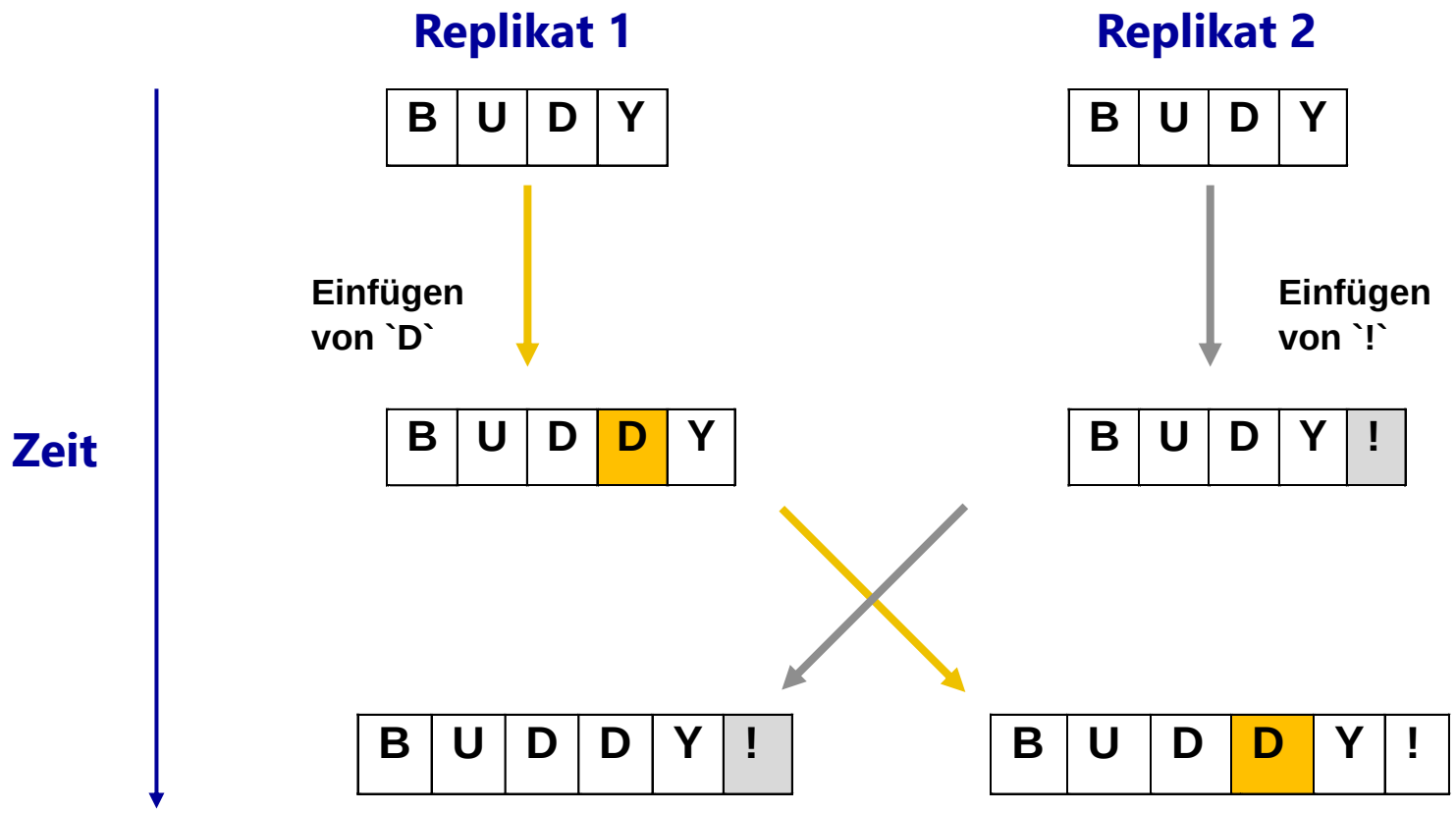
Textbearbeitung

Szenario: Zwei Benutzer ändern dasselbe Dokument mit dem Start „Budy“.



Textbearbeitung

Szenario: Zwei Benutzer ändern dasselbe Dokument mit dem Start „Budy“.



Algorithmen der Konvergenz

Konvergenz

Konvergenz liefert die Korrektheit innerhalb der Algorithmen und bedeutet, dass sich zwei Replikate im gleichen Zustand befinden, selbst wenn Aktualisierungen in unterschiedlicher Reihenfolge stattfinden.

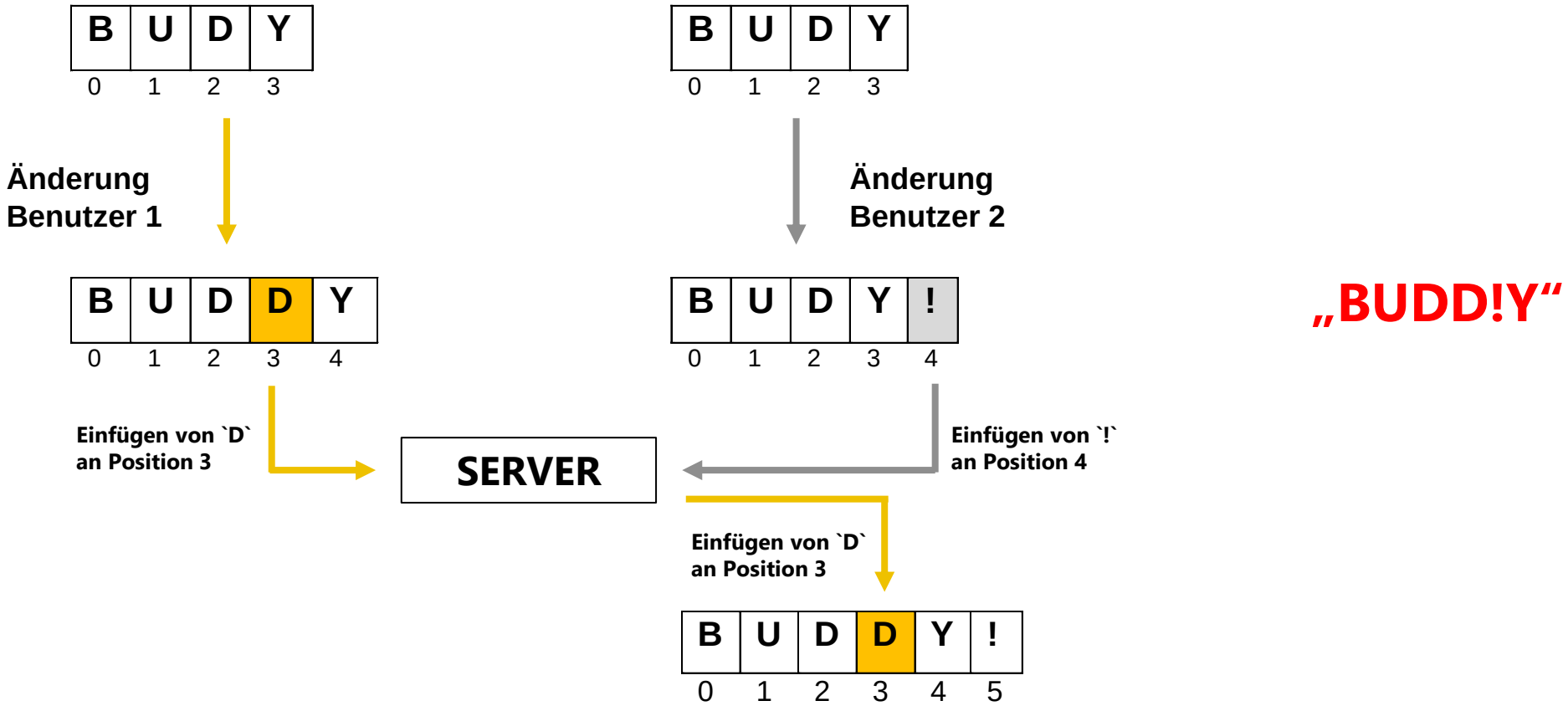
Operational Transformation (OT)

- Verwendet bis 2006
- Von Google Docs im Einsatz
- Kommunikation über einen zentralen Server

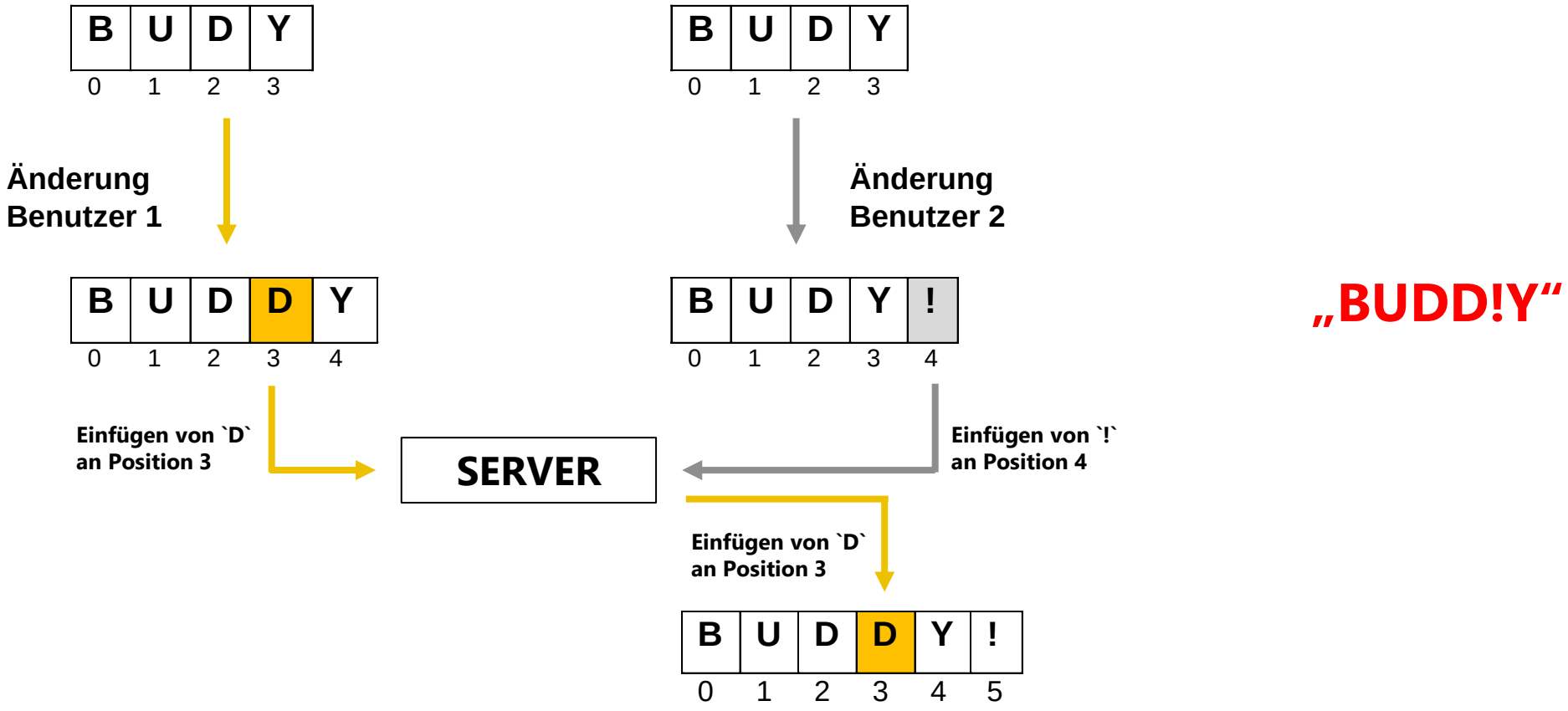
CRDT

- Verwendung ab 2006
- Alle Arten von Kommunikationskanäle
- Operationen werden kommutativ

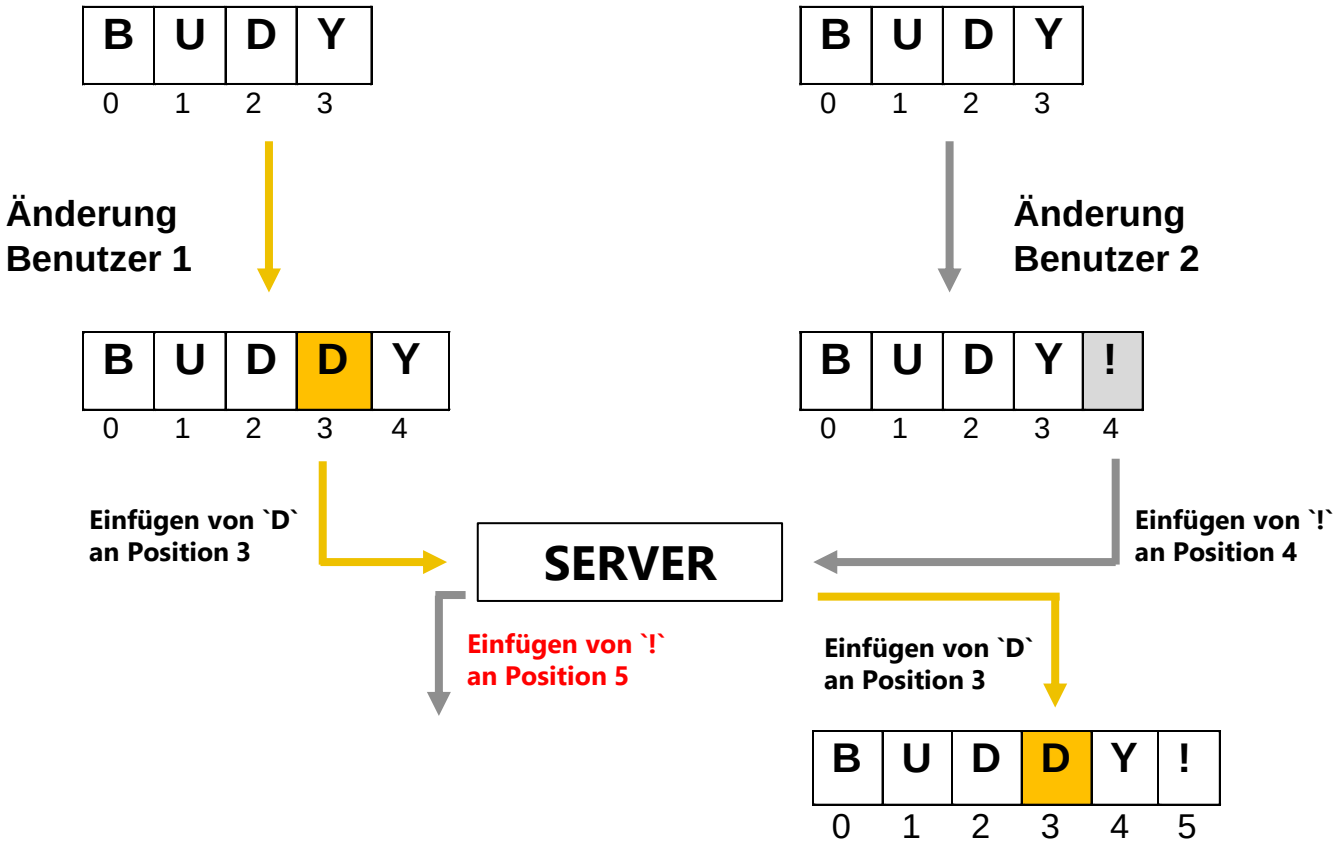
Text-Editing (OT)



Text-Editing (OT)



Text-Editing (OT)



Algorithmen der Konvergenz

Konvergenz

Konvergenz liefert die Korrektheit innerhalb der Algorithmen und bedeutet, dass sich zwei Replikate im gleichen Zustand befinden, selbst wenn Aktualisierungen in unterschiedlicher Reihenfolge stattfinden.

Operational Transformation (OT)

- Verwendet bis 2006
- Von Google Docs im Einsatz
- Kommunikation über einen zentralen Server

CRDT

- Verwendung ab 2006
- Alle Arten von Kommunikationskanäle
- Operationen werden kommutativ

Einführung in CRDTs

- CRDT ist eine Datenstruktur
- Ermöglicht Multiuser-Kollaboration

	Mobile Anwendungen	Verteilte Datenbanken	Software für Zusammenarbeit
Datensynchronisation	Speichern Daten auf dem lokalen Gerät und synchronisieren diese Daten mit anderen Geräten desselben Benutzers (z. B. Kalender, Notizen, Kontakte oder Erinnerungen).	Unterhalten mehrere Replikate der Daten (im selben Rechenzentrum oder an verschiedenen Standorten), so dass das System weiterhin korrekt funktioniert, wenn einige der Replikate offline sind.	Wie bei Google Docs können mehrere Benutzer gleichzeitig Änderungen an derselben Datei oder denselben Daten vornehmen.

1. Wie heißen zwei Algorithmen der Konvergenz?

2. Welche Grundannahme verfolgt der Algorithmus OT?

Arten von CRDTs

State Based Replication

- Update des States eines Objekts
- Lokal auf einem Replikat
- Der gesamte State wird über ein Netzwerk an alle anderen Replikate gesendet
- Der empfangene State wird durch ein Replikat zusammengeführt (Merge-Methode)

Operation Based Replication

- Es werden Operationen repliziert, kein State
- Jede Änderung wird als Vorgang codiert
- Gleichzeitige Operationen müssen kommutativ sein
- Änderungen werden in einer bestimmten Reihenfolge (häufig kausal) übermittelt

➡ **State-Based gilt als weniger komplex: Die Reihenfolge der Nachrichten ist egal. Alle Änderung können lokal betrachtet werden.**

Funktionsweise von CRDTs

Erinnerung: Bei OT wird ein Zeichen per Index eingefügt: Gezählt wird dabei immer ab Beginn des Dokuments

Merkmale:

- Vergabe einer UID
- UID ist einer Position im Dokument dauerhaft zugeteilt
- Vergabe kann durch Berechnung erfolgen (Mittelwert zweier Positionen)

Vorgehensweise (Beispiel):

1. Definition von Start und Ende
2. Zeichen einer Zahl im Intervall zuordnen
3. Bei Einfüge-Operationen kann der Mittelwert berechnet werden

B	U	D	Y
0.2	0.4	0.6	0.8

= {(0.2, X, ´B´), (0.4, X, ´U´), (0.6, X, ´D´), (0.8, X, ´Y´)}

Änderung

= {(0.7, Y, ´D´)}

B	U	D	D	Y
0.2	0.4	0.6	0.7	0.8

= {(0.2, X, ´B´), (0.4, X, ´U´), (0.6, X, ´D´), (0.7, Y, ´D´), (0.8, X, ´Y´)}

Herausforderungen bei der Implementierung von CRDTs

a

Verschachtelungen & Anomalien
Beim Zusammenführen der Operationen zweier Replikate entstehen Verschachtelungen

b

Verschieben von Elementen
Neue Anordnung von Elementen in Listen

c

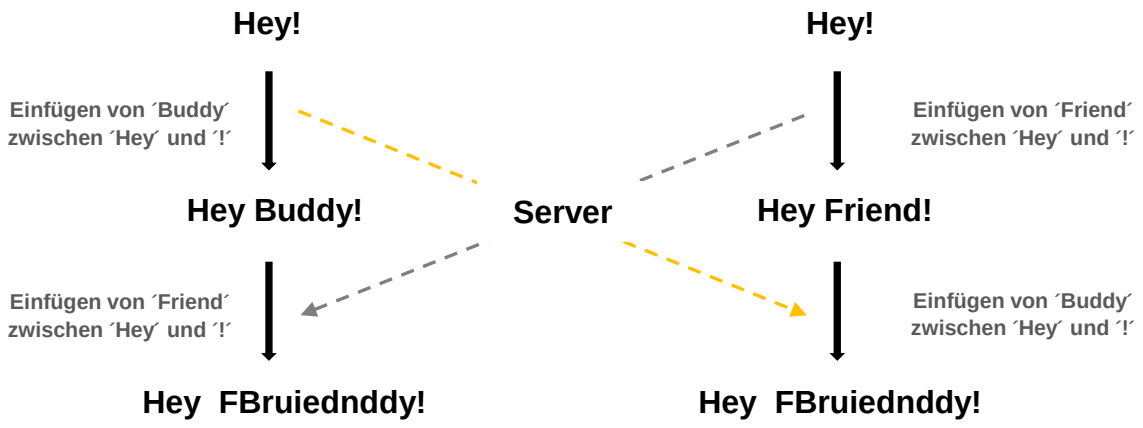
Verschieben von Baumstrukturen
Operationen innerhalb von Bäumen durch Verschieben von Knoten (Kinder)

d

Performance von CRDT Algorithmen
Geschwindigkeit der Durchführung von Verschiebe-Operationen

Verschachtelungen & Anomalien

Funktionsweise von CRDTs



H	e	y	!
0.23	0.47	0.72	0.97

H	e	y		B	u	d	d	y	!
0.23	0.47	0.72	0.76	0.80	0.84	0.88	0.92	0.96	0.97

H	e	y		F	r	i	e	n	d	!
0.23	0.47	0.72	0.75	0.78	0.81	0.85	0.87	0.90	0.93	0.97

H	e	y			F	B	r	u	i	e	d	n	d	d	y	!
0.23	0.47	0.72	0.75	0.76	0.78	0.80	0.81	0.84	0.85	0.87	0.88	0.90	0.92	0.93	0.96	0.97

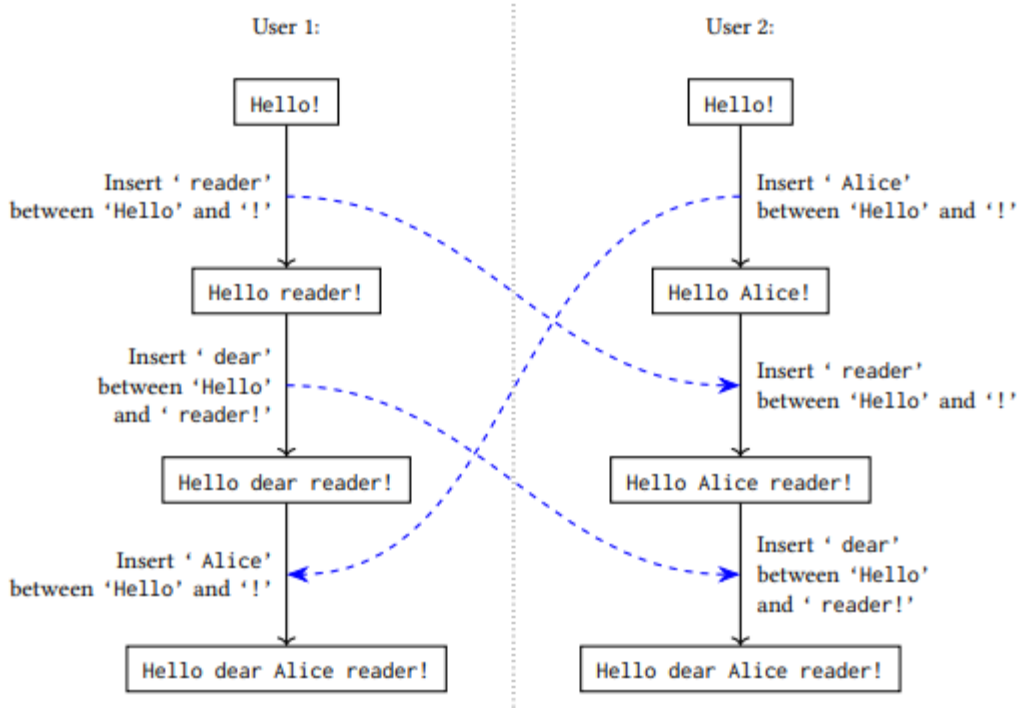
Quelle: https://martin.kleppmann.com/papers/interleaving-papoc19.pdf, S.2

Text-Editing CRDTs

1.	Logoot	Verschachtelung
2.	LSEQ	Verschachtelung
3.	WOOT	Verschachtelung
4.	Treedoc	Verschachtelung
5.	RGA	Weniger Anomalie
6.	Yjs	Weniger Anomalie
7.	Fugue	Keine Verschachtelung

Textbearbeitung CRDTs

Beispiel Verschachtelung anhand des Algorithmus **RGA**



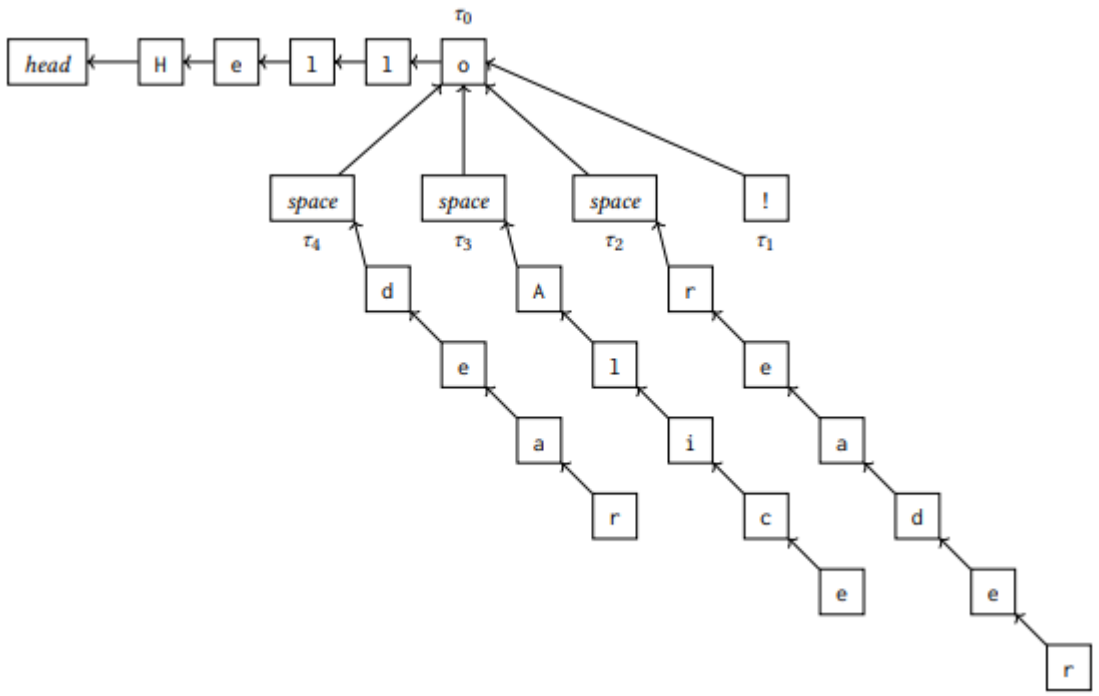
➡ Keine Character-After-Character-Verschachtelung.
Dieses Verhalten resultiert aus der Datenstruktur von RGA.

Textbearbeitung CRDTs

Datenstruktur von RGA

- Baumstruktur
- Jedes eingefügte Zeichen bildet einen Knoten
- Übergeordneter Knoten ist Vorgänger
- Jede Verschiebung des Cursor bildet einen neuen Teilbaum

➡ Die Reihenfolge wird durch Zeitstempel festgelegt



1. Bei welcher Art von CRDT-Algorithmus ist die Reihenfolge der Übertragung egal?

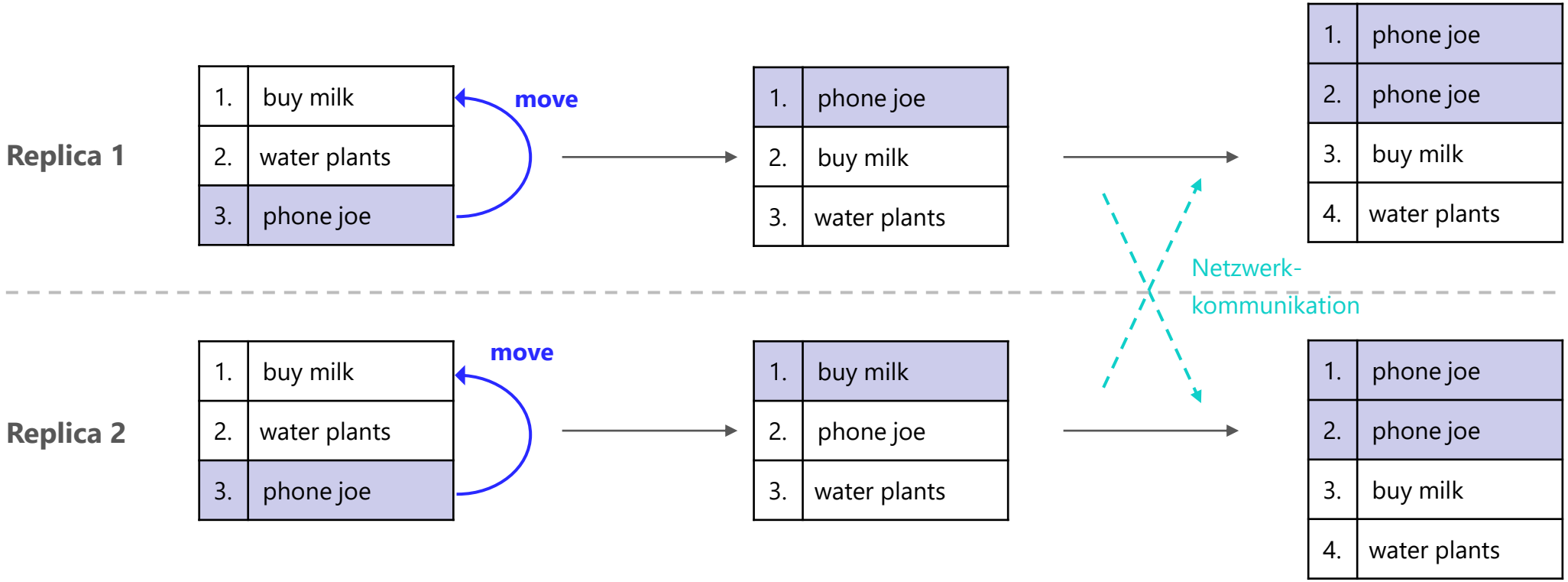
2. Wie kann eine Position im Dokument dauerhaft identifiziert werden. Welche Bestandteile gibt es?

Verschiebung von Elementen in Listen

Verschiebung von Elementen in Listen

Beispiel einer ToDo-Liste mit Drag&Drop-Funktion

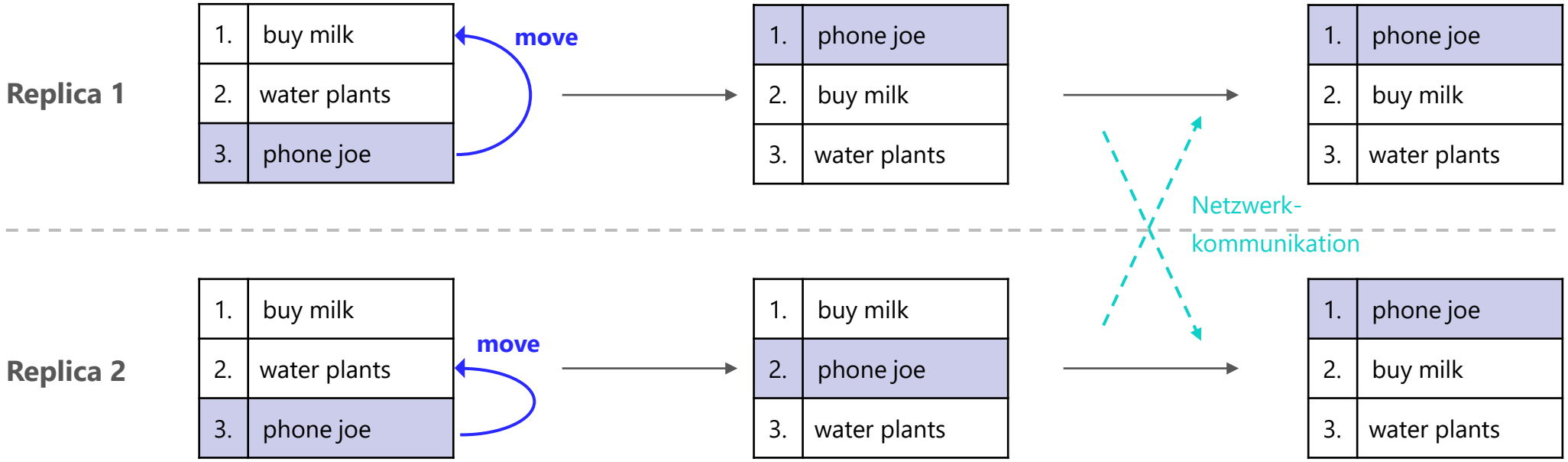
- CRDTs unterstützen keine Verschiebe-Operationen
- Workaround: Alte Position löschen, neue Position einfügen
- **Problem 1: Gleichzeitige Änderung derselben Position**



Verschiebung von Elementen in Listen

Beispiel einer ToDo-Liste mit Drag&Drop-Funktion

- CRDTs unterstützen keine Verschiebe-Oprationen
- Workaround: Alte Position löschen, neue Position einfügen
- **Problem 2: Gleichzeitige Änderung an verschiedene Positionen**



Verschiebung von Elementen in Listen

Lösung für Problem 1 & 2

Last Writer Wins (LWW) Register

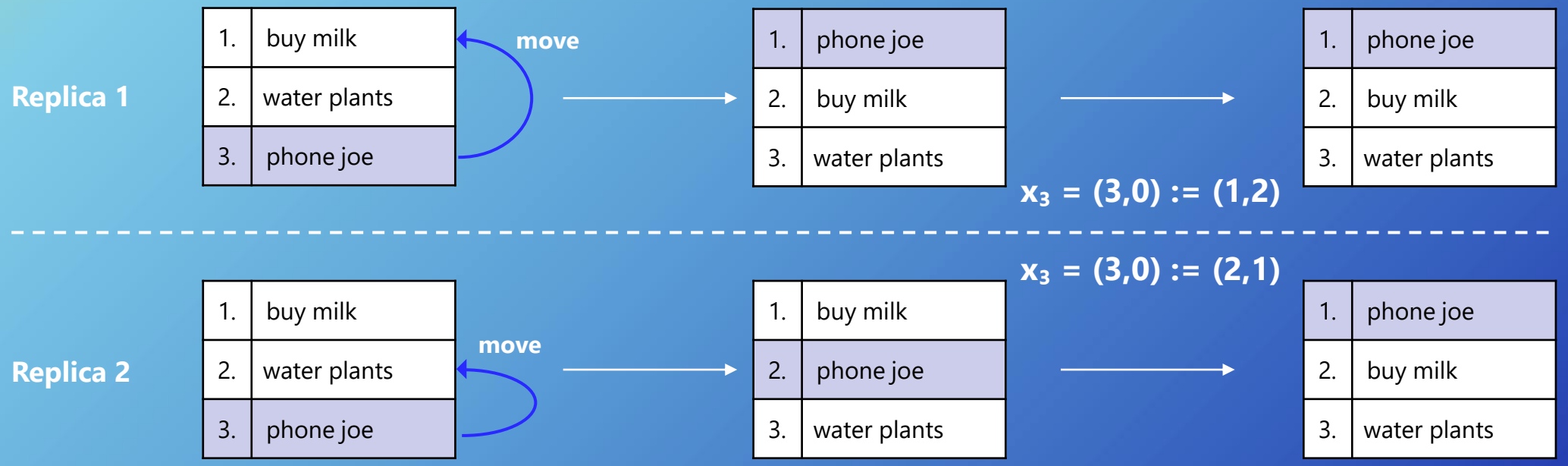
- Ein Register ist eine Speicherzelle
- Ein LWW-Register bevorzugt eine Aktualisierung und ignoriert die andere
- Dafür registriert eine Variable den Zeitstempel und kann diesen abfragen
- Jedes Element in der Liste erhält einen LWW-Register
- Der Inhalt des LWW-Register ist die Position des Elements

```
payload X x, timestamp t ◃ X: some type
        initial ⊥, 0
update assign (X w)
        x, t := w, now() ◃ Timestamp, consistent with causality
query value () : X w
        let w = x
compare (R, R′) : boolean b
        let b = (R.t ≤ R′.t)
merge (R, R′) : payload R′′
        if R.t ≤ R′ .t then R ′′.x, R′′.t = R′.x, R′.t
        else R′′.x, R′′.t = R.x, R.t
```

Verschiebung von Elementen in Listen

Lösung für Problem 1 & 2

Last Writer Wins (LWW) Register

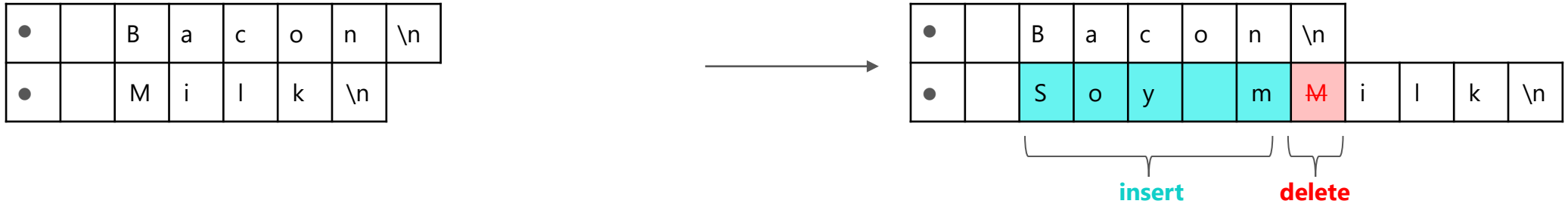


Verschiebung von Elementen in Listen

Beispiel einer Einkaufsliste als Klartext

- Elemente beginnen mit einem Aufzählungszeichen und enden mit neuer Zeile
- Gesamter Bereich wird verschoben
- **Problem 3: Gleichzeitiges Verschieben von Bereichen**

Replica 1

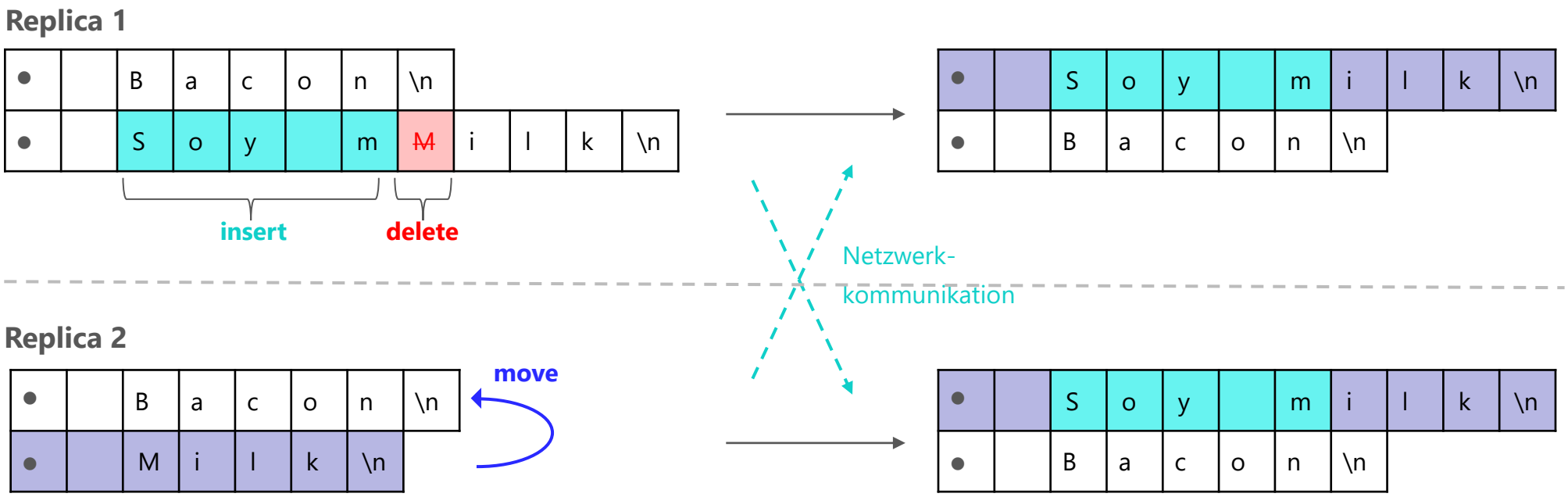


Replica 2

Verschiebung von Elementen in Listen

Beispiel einer Einkaufsliste als Klartext

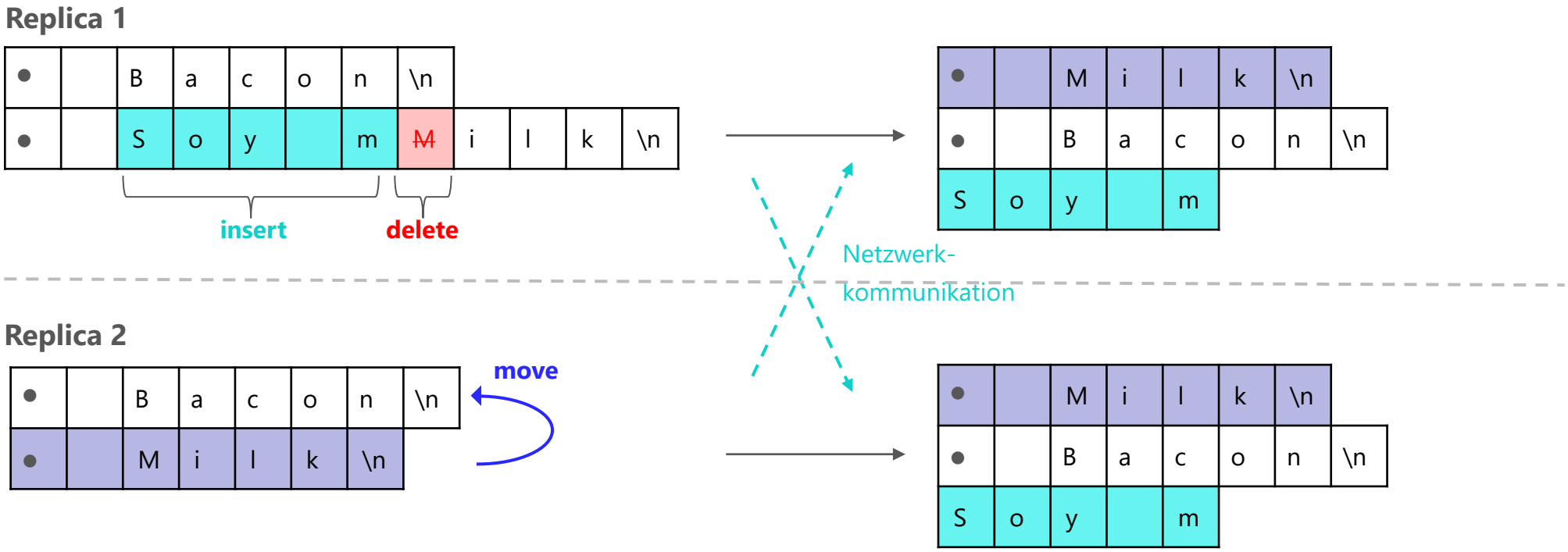
- Elemente beginnen mit einem Aufzählungszeichen und enden mit neuer Zeile
- Gesamter Bereich wird verschoben
- **Problem 3: Gleichzeitiges Verschieben von Bereichen**



Verschiebung von Elementen in Listen

Beispiel einer Einkaufsliste als Klartext

- Elemente beginnen mit einem Aufzählungszeichen und enden mit neuer Zeile
- Gesamter Bereich wird verschoben
- **Problem 3: Gleichzeitiges Verschieben von Bereichen**



Verschiebung von Elementen in Listen

Lösung für Problem 3

Problem besteht weiterhin:

“At present, there is no known algorithm for performing such moves of ranges of elements. Various tricky edge-cases would need to be handled by such an algorithm, such as concurrent moves of partially overlapping ranges, and moves whose destination position falls within another range that is itself being moved.” – April 2020

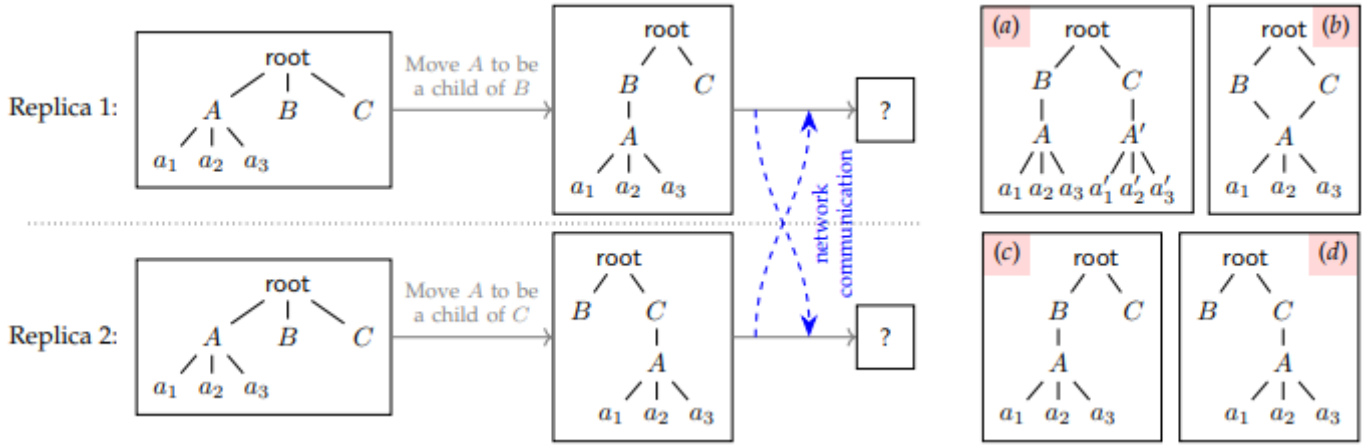
Steven Johnston https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 4

Verschiebung von Knoten in Baumstrukturen

Verschiebung von Knoten in Baumstrukturen

Beispiel einer Baumstruktur (Dateisystem)

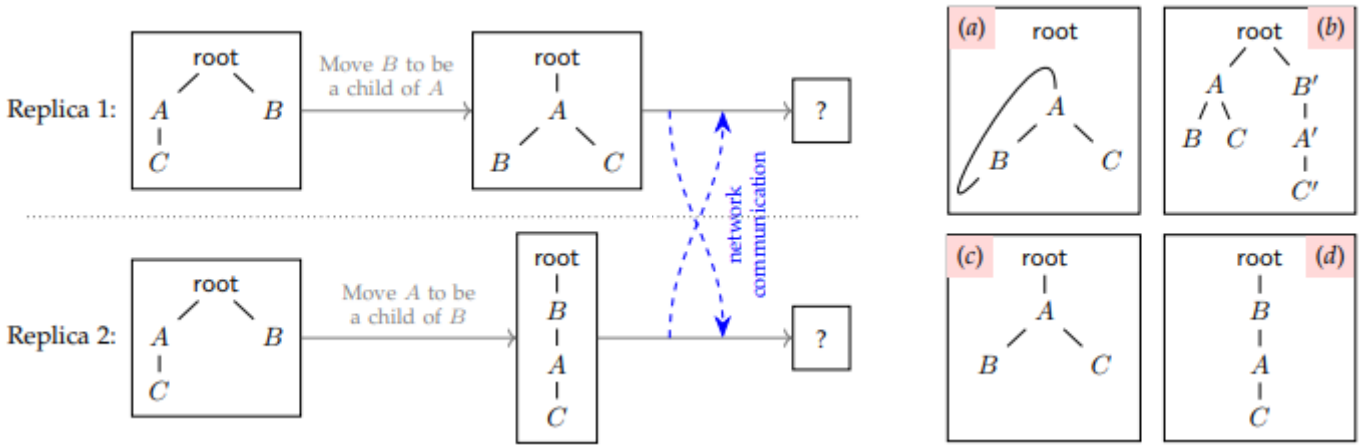
- Verschiebung desselben Knotens führt zu unterschiedlichen Ergebnissen
- **Problem 4:** **Gleichzeitiges Verschieben von Knoten**
  Mögliche Ergebnisse wären (a) Duplikate, (b) keine Baumstruktur, (c) und (d) LWW



Verschiebung von Knoten in Baumstrukturen

Beispiel einer Baumstruktur (Dateisystem)

- Verschiebung desselben Knotens führt zu unterschiedlichen Ergebnissen
- Beide Operationen einzeln betrachtet sind zulässig
- **Problem 5:** **Kombination aus gleichzeitigem Verschieben von Knoten**
  (a) rekursiv (getrennt von root), (b) Duplikate, (c) und (d) LWW



Verschiebung von Knoten in Baumstrukturen

Lösung für Problem 5

Merge-Operation:

- Operationen werden in Sequenzen eingeteilt
- Zeitstempel ermöglicht das Zusammenführen
- Ergebnis ist eine einzige Sequenz in Reihenfolge der Zeitstempel

Herausforderung:

Operationen von einem Replikat X ts(op1, op3, op5) wurden bereits ausgeführt, während Replikat Y ts(op2) sendet.

```
fun undo_op :: ('t, 'n, 'm) log_op × ('n × 'm × 'n) set ⇒ ('n × 'm × 'n) set where
  undo_op (LogMove t None newp m c, tree) = {(p', m', c') ∈ tree. c' ≠ c} |
  undo_op (LogMove t (Some (oldp, oldm)) newp m c, tree) =
    {(p', m', c') ∈ tree. c' ≠ c} ∪ {(oldp, oldm, c)}

fun redo_op :: ('t, 'n, 'm) log_op ⇒ ('t, 'n, 'm) state ⇒ ('t, 'n, 'm) state where
  redo_op (LogMove t _ p m c) (ops, tree) =
    (let (op2, tree2) = do_op (Move t p m c, tree)
     in (op2 # ops, tree2))

fun apply_op :: ('t::{linorder}, 'n, 'm) operation ⇒ ('t, 'n, 'm) state ⇒ ('t, 'n, 'm) state where
  apply_op op1 ([], tree1) =
    (let (op2, tree2) = do_op (op1, tree1)
     in ([op2], tree2)) |
  apply_op op1 (logop # ops, tree1) =
    (if move_time op1 < log_time logop
     then redo_op logop (apply_op op1 (ops, undo_op (logop, tree1)))
     else let (op2, tree2) = do_op (op1, tree1) in (op2 # logop # ops, tree2))
```

Verschiebung von Knoten in Baumstrukturen

Replica A

timestamps

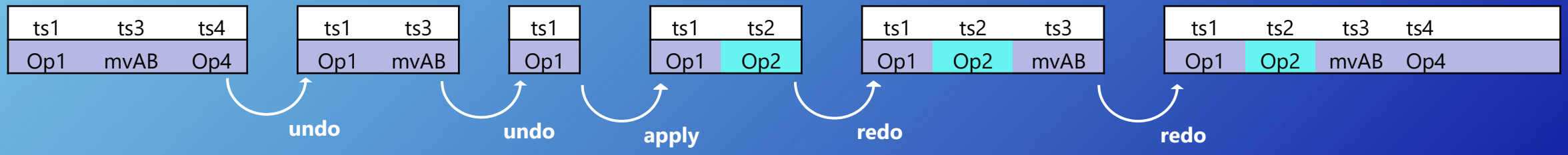
ts1	ts3	ts4
Op1	mvAB	Op4

Replica B

timestamps

ts2	ts5	ts6
Op2	mvBA	Op6

ts1	ts2	ts3	ts4	ts5	ts6
Op1	Op2	mvAB	Op4	mvBA	Op6



Performance in CRDT Algorithmen

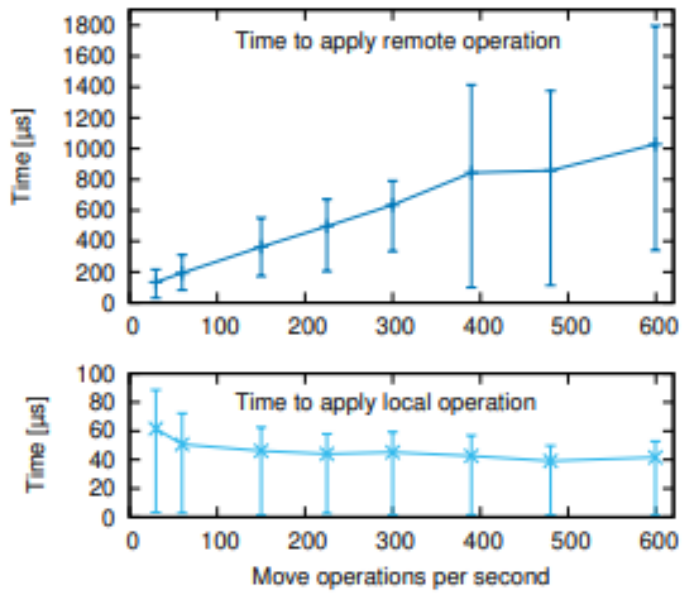
Performance in CRDT Algorithmen

Problem aus der Logik zuvor:
Was tun wir, wenn eine Operation sehr früh in einer Sequenz eingefügt werden muss?

➡ Viele `undo_ops` und `redo_ops` entstehen

Experiment

- Drei Replikate (1) Kalifornien, (2) Irland, (3) Singapur tätigen Operationen
- Die Netzwerkverzögerung von Kalifornien zu Irland beträgt 145ms, von Singapur nach Kalifornien 176ms.
- Ein Replikat wendet eine Operation zunächst lokal an und sendet sie asynchron an die beiden anderen Replikate.
- Das Ergebnis des Experiments ist, dass 600 Operationen/Sek. durchgeführt werden können.



Zusammenfassung

Zusammenfassung

1. CRDTs ermöglichen replizierte Datenstrukturen ohne Kommunikation über einen zentrale Server.
2. Es treten Konflikte auf, wenn mehrere Benutzer gleichzeitig an verschiedenen Replikaten arbeiten.
3. Bei fast allen vorgestellten Algorithmen gibt es Verschachtelungen & Anomalien.
4. Herausforderungen bei CRDT-Algorithmen:
    a. CRDT-Algorithmen können relativ einfach implementiert werden. Das Resultat sollte ein für den Menschen sinnvolles Konstrukt ergeben.
    b. Einen gut konzipierten CRDT-Algorithmen für komplexe Strukturen wie Listen und Bäume zu bauen ist eine schwierige Aufgabe.
    c. Es gibt Probleme bei Verschiebe-Operationen, für die noch keine Lösung bereitgestellt wurde.
5. Die Algorithmen müssen effizient designt werden, um eine gute Systemperformance gewährleisten zu können.

Übung

Rufen Sie den folgenden Link auf